4/Variables

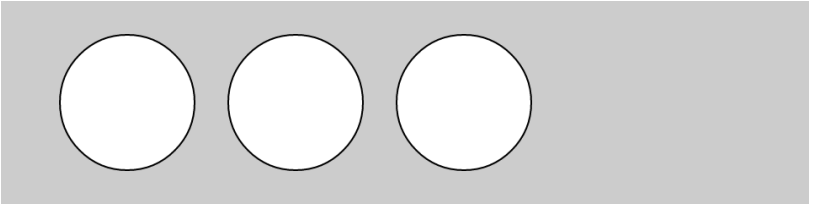
A *variable* stores a value in memory so that it can be used later in a program. A variable can be used many times within a single program, and the value is easily changed while the program is running.

First Variables

The primary reason we use variables is to avoid repeating ourselves in the code. If you are typing the same number more than once, consider using a variable instead so that your code is more general and easier to update.

Example 4-1: Reuse the Same Values

For instance, when you make the *y* coordinate and diameter for the three circles in this example into variables, the same values are used for each ellipse:



```
var y = 60;
var d = 80;

function setup() {
  createCanvas(480, 120);
}

function draw() {
```

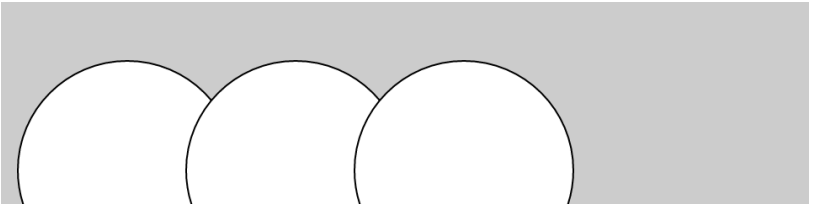
Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing, Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

```
  background(204);
  ellipse(75, y, d, d);   // Left
  ellipse(175, y, d, d);  // Middle
  ellipse(275, y, d, d);  // Right
}
```

Example 4-2: Change Values

Simply changing the *y* and *d* variables therefore alters all three ellipses:



```
var y = 100;
var d = 130;

function setup() {
  createCanvas(480, 120);
}

function draw() {
  background(204);
  ellipse(75, y, d, d);   // Left
  ellipse(175, y, d, d);  // Middle
  ellipse(275, y, d, d);  // Right
}
```

Without the variables, you'd need to change the *y* coordinate used in the code three times and the diameter six times. When comparing Examples 4-1 and 4-2, notice how all the lines are the same, except the first two lines with the variables are different. Variables allow you to separate the lines of the code that change from the lines that don't, which makes programs easier to modify. For instance, if you place variables that control colors and sizes of shapes in one place, then you can quickly explore different visual options by focusing on only a few lines of code.

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

Making Variables

When you make your own variables, you determine the *name* and the *value*. The name is what you decide to call the variable. Choose a name that is informative about what the variable stores, but be consistent and not too verbose. For instance, the variable name "radius" will be clearer than "r" when you look at the code later.

Variables must first be *declared*, which sets aside space in the computer's memory to store the information. When declaring a variable, you use `var`, to indicate you are creating a new variable, followed by the name. After the name is set, a value can be assigned to the variable:

```
var x;  // Declare x as a variable
x = 12; // Assign a value to x
```

This code does the same thing, but is shorter:

```
var x = 12; // Declare x as a variable and assign a value
```

The characters `var` are included on the line of code that declares a variable, but they're not written again. Each time `var` is written in front of the variable name, the computer thinks you're trying to declare a new variable. You can't have two variables with the same name in the same part of the program (Appendix C), or the program could behave strangely:

```
var x;       // Declare x as a variable
var x = 12;  // ERROR! Can't have two variables called x here
```

You can place your variables outside of `setup()` and `draw()`. If you create a variable inside of `setup()`, you can't use it inside of `draw()`, so you need to place those variables somewhere else. Such variables are called *global variables*, because they can be used anywhere ("globally") in the program.

p5.js Variables

p5.js has a series of special variables to store information about the program while it runs. For instance, the width and height of the canvas are stored in variables called `width` and `height`. These values are set by the `createCanvas()` function. They can

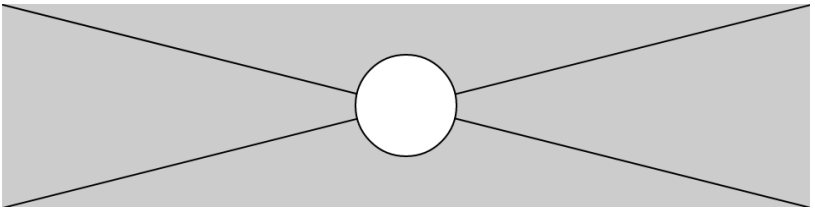
Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

be used to draw elements relative to the size of the canvas, even if the `createCanvas()` line changes.

Example 4-3: Adjust the Canvas, See What Follows

In this example, change the parameters to `createCanvas()` to see how it works:



```
function setup() {
  createCanvas(480, 120);
}

function draw() {
  background(204);
  line(0, 0, width, height);  // Line from (0,0) to (480, 120)
  line(width, 0, 0, height);  // Line from (480, 0) to (0, 120)
  ellipse(width/2, height/2, 60, 60);
}
```

Other special variables keep track of the status of the mouse and keyboard values and much more. These are discussed in Chapter 5.

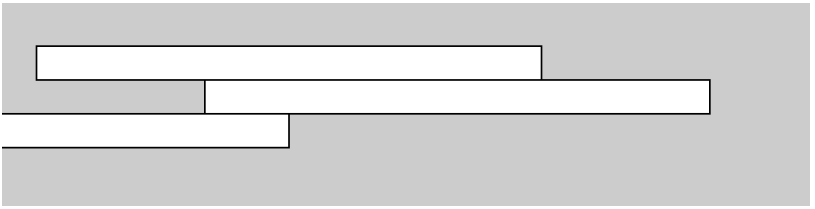
A Little Math

People often assume that math and programming are the same thing. Although knowledge of math can be useful for certain types of coding, basic arithmetic covers the most important parts.

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

Example 4-4: Basic Arithmetic



```
var x = 25;
var h = 20;
var y = 25;

function setup() {
  createCanvas(480, 120);
}

function draw() {
  background(204);
  x = 20;
  rect(x, y, 300, h);        // Top
  x = x + 100;
  rect(x, y + h, 300, h);    // Middle
  x = x - 250;
  rect(x, y + h*2, 300, h);  // Bottom
}
```

In code, symbols like +, -, and * are called *operators*. When placed between two values, they create an *expression*. For instance, 5 + 9 and 1024 – 512 are both expressions. The operators for the basic math operations are:

+	Addition
–	Subtraction
*	Multiplication
/	Division
=	Assignment

JavaScript has a set of rules to define which operators take precedence over others, meaning which calculations are made first, second, third, and so on. These rules define the order in which the code is run. A little knowledge about this goes a long

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

way toward understanding how a short line of code like this works:

```
var x = 4 + 4 * 5; // Assign 24 to x
```

The expression 4 * 5 is evaluated first because multiplication has the highest priority. Second, 4 is added to the product of 4 * 5 to yield 24. Last, because the *assignment operator* (the *equals* sign) has the lowest precedence, the value 24 is assigned to the variable *x*. This is clarified with parentheses, but the result is the same:

```
var x = 4 + (4 * 5); // Assign 24 to x
```

If you want to force the addition to happen first, just move the parentheses. Because parentheses have a higher precedence than multiplication, the order is changed and the calculation is affected:

```
var x = (4 + 4) * 5; // Assign 40 to x
```

An acronym for this order is often taught in math class: PEMDAS, which stands for Parentheses, Exponents, Multiplication, Division, Addition, Subtraction, where parentheses have the highest priority and subtraction the lowest. The complete order of operations is found in Appendix B.

Some calculations are used so frequently in programming that shortcuts have been developed; it's always nice to save a few keystrokes. For instance, you can add to a variable, or subtract from it, with a single operator:

```
x += 10; // This is the same as x = x + 10
y -= 15; // This is the same as y = y - 15
```

It's also common to add or subtract 1 from a variable, so shortcuts exist for this as well. The `++` and `--` operators do this:

```
x++; // This is the same as x = x + 1
y--; // This is the same as y = y - 1
```

Repetition

As you write more programs, you'll notice that patterns occur when lines of code are repeated, but with slight variations. A code structure called a `for` loop makes it possible to run a line of

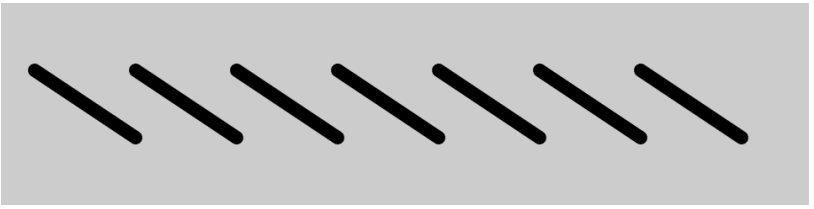
Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

code more than once to condense this type of repetition into fewer lines. This makes your programs more modular and easier to change.

Example 4-5: Do the Same Thing Over and Over

This example has the type of pattern that can be simplified with a `for` loop:



```
function setup() {
  createCanvas(480, 120);
  strokeWeight(8);
}

function draw() {
  background(204);
  line(20, 40, 80, 80);
  line(80, 40, 140, 80);
  line(140, 40, 200, 80);
  line(200, 40, 260, 80);
  line(260, 40, 320, 80);
  line(320, 40, 380, 80);
  line(380, 40, 440, 80);
}
```

Example 4-6: Use a for Loop

The same thing can be done with a `for` loop, and with less code:

```
function setup() {
  createCanvas(480, 120);
  strokeWeight(8);
}

function draw() {
  background(204);
  for (var i = 20; i < 400; i += 60) {
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

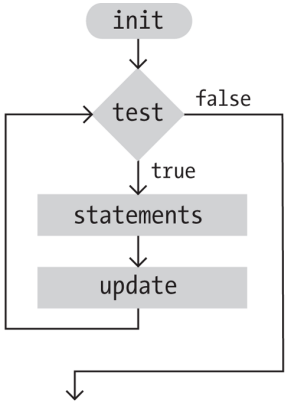
```
    line(i, 40, i + 60, 80);
  }
}
```

The `for` loop is different in many ways from the code we've written so far. Notice the braces, the { and } characters. The code between the braces is called a *block*. This is the code that will be repeated on each iteration of the `for` loop.

Inside the parentheses are three statements, separated by semicolons, that work together to control how many times the code inside the block is run. From left to right, these statements are referred to as the *initialization* (`init`), the *test*, and the *update*:

```
for (init; test; update) {
  statements
}
```

The `init` typically declares a new variable to use within the `for` loop and assigns a value. The variable name `i` is frequently used, but there's really nothing special about it. The *test* evaluates the value of this variable, and the *update* changes the variable's value. Figure 4-1 shows the order in which they run and how they control the code statements inside the block.




```
for (init; test; update) {
  statements
}
```

Figure 4-1. *Flow diagram of a for loop*

The *test* statement requires more explanation. It's always a *relational expression* that compares two values with a *relational operator*. In this example, the expression is "i < 400" and the

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

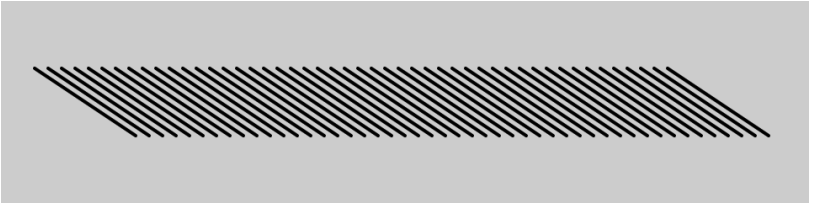
operator is the < (less than) symbol. The most common relational operators are:

>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

The relational expression always evaluates to `true` or `false`. For instance, the expression 5 > 3 is `true`. We can ask the question, "Is five greater than three?" Because the answer is "yes," we say the expression is `true`. For the expression 5 < 3, we ask, "Is five less than three?" Because the answer is "no," we say the expression is `false`. When the evaluation is `true`, the code inside the block is run, and when it's `false`, the code inside the block is not run and the `for` loop ends.

Example 4-7: Flex Your for Loop's Muscles

The ultimate power of working with a `for` loop is the ability to make quick changes to the code. Because the code inside the block is typically run multiple times, a change to the block is magnified when the code is run. By modifying Example 4-6 on page 47 only slightly, we can create a range of different patterns:



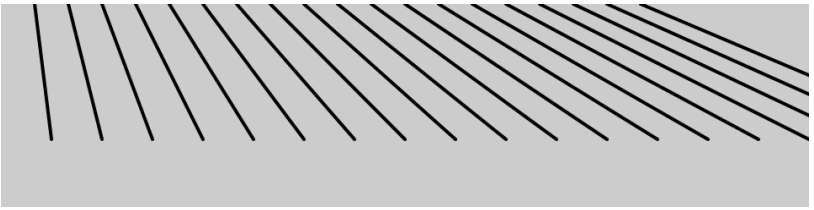
```
function setup() {
  createCanvas(480, 120);
  strokeWeight(2);
}
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

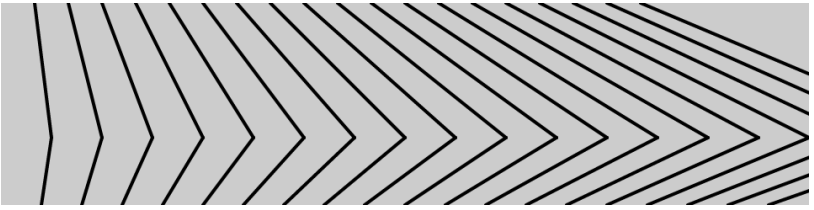
```
function draw() {
  background(204);
  for (var i = 20; i < 400; i += 8) {
    line(i, 40, i + 60, 80);
  }
}
```

Example 4-8: Fanning Out the Lines



```
function setup() {
  createCanvas(480, 120);
  strokeWeight(2);
}

function draw() {
  background(204);
  for (var i = 20; i < 400; i += 20) {
    line(i, 0, i + i/2, 80);
  }
}
```

Example 4-9: Kinking the Lines



```
function setup() {
  createCanvas(480, 120);
  strokeWeight(2);
}
```

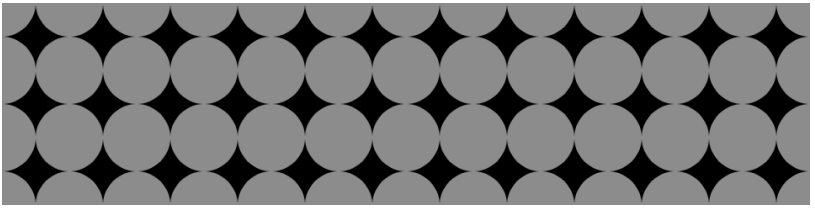
Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

```
function draw() {
  background(204);
  for (var i = 20; i < 400; i += 20) {
    line(i, 0, i + i/2, 80);
    line(i + i/2, 80, i*1.2, 120);
  }
}
```

Example 4-10: Embed One for Loop in Another

When one `for` loop is embedded inside another, the number of repetitions is multiplied. First, let's look at a short example, and then we'll break it down in Example 4-11 on page 51:



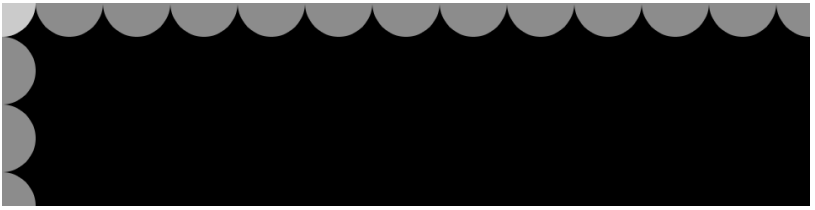
```
function setup() {
  createCanvas(480, 120);
  noStroke();
}

function draw() {
  background(0);
  for (var y = 0; y <= height; y += 40) {
    for (var x = 0; x <= width; x += 40) {
      fill(255, 140);
      ellipse(x, y, 40, 40);
    }
  }
}
```

Example 4-11: Rows and Columns

In this example, the `for` loops are adjacent, rather than one embedded inside the other. The result shows that one `for` loop is drawing a column of 4 circles and the other is drawing a row of 13 circles:

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.



```
function setup() {
  createCanvas(480, 120);
  noStroke();
}

function draw() {
  background(0);
  for (var y = 0; y < height+45; y += 40) {
    fill(255, 140);
    ellipse(0, y, 40, 40);
  }
  for (var x = 0; x < width+45; x += 40) {
    fill(255, 140);
    ellipse(x, 0, 40, 40);
  }
}
```

When one of these `for` loops is placed inside the other, as in Example 4-10 on page 51, the 4 repetitions of the first loop are compounded with the 13 of the second in order to run the code inside the embedded block 52 times (4×13 = 52).

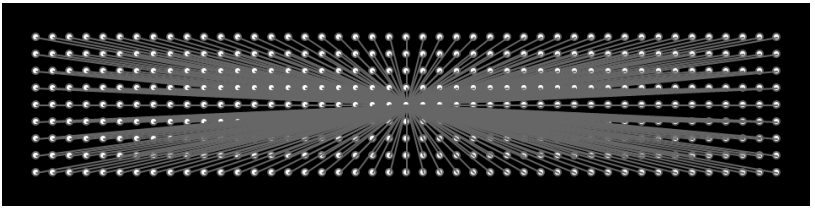
Example 4-10 on page 51 is a good base for exploring many types of repeating visual patterns. The following examples show a couple of ways that it can be extended, but this is only a tiny sample of what's possible.

Example 4-12: Pins and Lines

In this example, the code draws a line from each point in the grid to the center of the screen:

Copyright © 2015. Maker Media, Inc. All rights reserved.

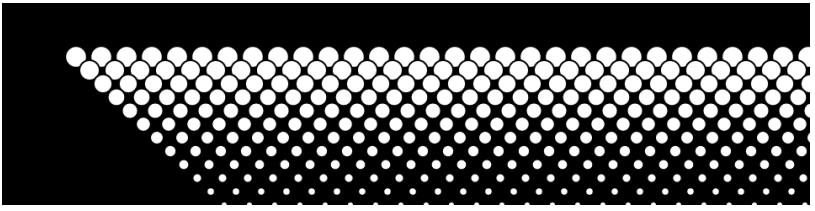
McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.



```
function setup() {
  createCanvas(480, 120);
  fill(255);
  stroke(102);
}

function draw() {
  background(0);
  for (var y = 20; y <= height-20; y += 10) {
    for (var x = 20; x <= width-20; x += 10) {
      ellipse(x, y, 4, 4);
      // Draw a line to the center of the display
      line(x, y, 240, 60);
    }
  }
}
```

Example 4-13: Halftone Dots

In this example, the ellipses shrink with each new row and are moved to the right by adding the *y* coordinate to the *x* coordinate:



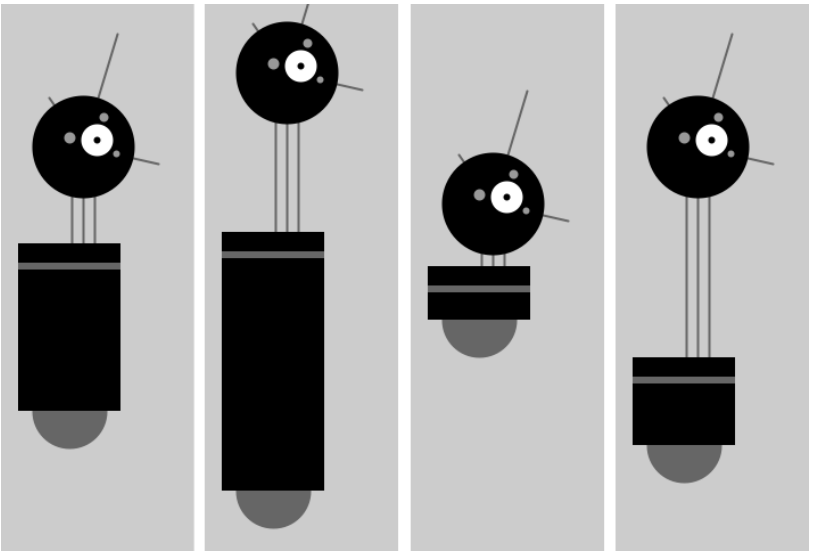
```
function setup() {
  createCanvas(480, 120);
}

function draw() {
  background(0);
  for (var y = 32; y <= height; y += 8) {
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

```
    for (var x = 12; x <= width; x += 15) {
      ellipse(x + y, y, 16 - y/10.0, 16 - y/10.0);
    }
  }
}
```

Robot 2: Variables



The variables introduced in this program make the code look more difficult than Robot 1 (see "Robot 1: Draw" on page 38), but now it's much easier to modify, because numbers that depend on one another are in a single location. For instance, the neck is drawn based on the `neckHeight` variable. The group of variables at the top of the code control the aspects of the robot that we want to change: location, body height, and neck height. You can see some of the range of possible variations in the figure; from left to right, here are the values that correspond to them:

y = 390 bodyHeight = 180 neckHeight = 40	y = 460 bodyHeight = 260 neckHeight = 95	y = 310 bodyHeight = 80 neckHeight = 10	y = 420 bodyHeight = 110 neckHeight = 140

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

When altering your own code to use variables instead of numbers, plan the changes carefully, then make the modifications in short steps. For instance, when this program was written, each variable was created one at a time to minimize the complexity of the transition. After a variable was added and the code was run to ensure it was working, the next variable was added:

```
var x = 60;             // x coordinate
var y = 420;            // y coordinate
var bodyHeight = 110;   // Body height
var neckHeight = 140;   // Neck height
var radius = 45;
var ny = y - bodyHeight - neckHeight - radius;  // Neck Y

function setup() {
  createCanvas(170, 480);
  strokeWeight(2);
  ellipseMode(RADIUS);
}

function draw() {
  background(204);

  // Neck
  stroke(102);
  line(x+2, y-bodyHeight, x+2, ny);
  line(x+12, y-bodyHeight, x+12, ny);
  line(x+22, y-bodyHeight, x+22, ny);

  // Antennae
  line(x+12, ny, x-18, ny-43);
  line(x+12, ny, x+42, ny-99);
  line(x+12, ny, x+78, ny+15);

  // Body
  noStroke();
  fill(102);
  ellipse(x, y-33, 33, 33);
  fill(0);
  rect(x-45, y-bodyHeight, 90, bodyHeight-33);
  fill(102);
  rect(x-45, y-bodyHeight+17, 90, 6);

  // Head
  fill(0);
  ellipse(x+12, ny, radius, radius);
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing, Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

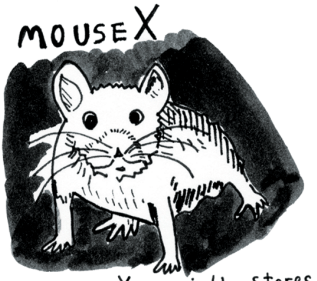
```
  fill(255);
  ellipse(x+24, ny-6, 14, 14);
  fill(0);
  ellipse(x+24, ny-6, 3, 3);
  fill(153);
  ellipse(x, ny-8, 5, 5);
  ellipse(x+30, ny-26, 4, 4);
  ellipse(x+41, ny+6, 3, 3);
}
```

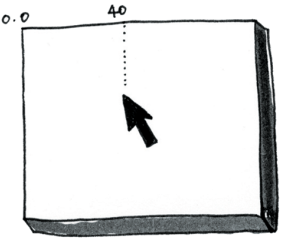
Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.

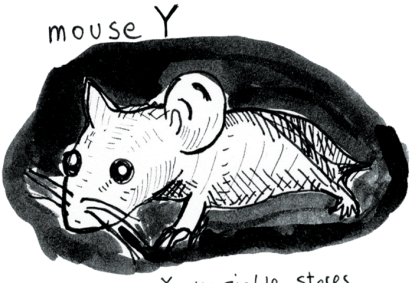
Copyright © 2015. Maker Media, Inc. All rights reserved.

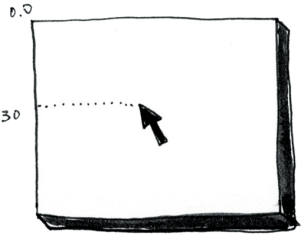
McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.



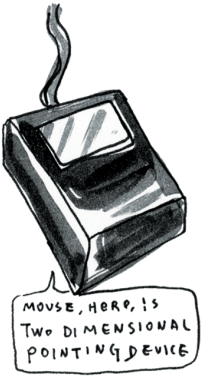


THE mouseX variable stores the X-coordinate





THE mouseY variable stores the Ycoordinate



Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-10 15:33:30.